

The Consistency Model in Apache Ratis

施子和

Sze, Tsz-Wo (Nicholas)

Principal Software Engineer / PMC Chair of Apache Ratis



2026.8.8

v20260809

About Me

施 子 和 Sze, Tsz-Wo (Nicholas), Ph.D.

- Software Engineer / Amateur Mathematician
- PMC member of Apache Ratis, Ozone & Hadoop

Fun facts

- Used Hadoop to create a **π computation record** (2010)
 - BBC News: <https://www.bbc.com/news/technology-11313194>
- Discovered a deterministic **primality proving algorithm** for Proth numbers (2008)
 - Wikipedia: https://en.wikipedia.org/wiki/Proth_prime#Primality_testing
 - The Online Encyclopedia of Integer Sequence: <https://oeis.org/A080076>

What did Google AI say?



Google

size tsz wo

AI Mode **All** Images Videos News Shopping Forums More Tools

 **Sze, Tsz-Wo**
<https://szetszwo.github.io>

Sze, Tsz-Wo (Nicholas)
Aug 22, 2025 — Tsz-Wo is the founder of **Apache Ratis**, which is an open source Java library for the RAFT Consensus Algorithm. He has been working on Apache Hadoop since 2007.

✦ **AI Overview** 

Tsz-Wo (Nicholas) Sze is a **computer scientist and software engineer** known for founding Apache Ratis and his work on Apache Hadoop. He earned his Ph.D. from the University of Maryland, College Park in 2007 and previously used Yahoo's computing infrastructure to compute distant digits of π .  Sze, Tsz-Wo +2

Background and Education

- Obtained a B.Eng. (1999) and M.Phil. (2001) from the Hong Kong University of Science and Technology.
- Completed his Ph.D. in Mathematics/Computer Science at the University of Maryland in 2007.  The Mathematics Genea... +1

Professional Work

- Active contributor and committer to Apache Hadoop since 2007.
- Founder of **Apache Ratis**, a Java library implementing the Raft consensus algorithm.
- Member of The Apache Software Foundation.  Sze, Tsz-Wo

Google

size tsz wo math

AI Mode **All** Images Videos Shopping Forums News More Tools

✦ **AI Overview**

Tsz-Wo Sze (also known as Nicholas Sze) is a **mathematician and computer scientist specializing in number theory, finite fields, and distributed algorithms**. He earned his Ph.D. in Computer Science from the University of Maryland, College Park in 2007 under advisor Lawrence C. Washington, and is widely known for his research in deterministic algorithms, primality testing, and distributed computations of constants like π .  The Mathematics Genea... +2

Academic and Research Highlights

- **Ph.D. Dissertation:** Authored *On Solving Univariate Polynomial Equations over Finite Fields and Some Related Problems* at the University of Maryland, focusing on deterministic polynomial-time algorithms for cryptography and number theory.  The Mathematics Genea... +1
- **Square Root Algorithms:** Published notable work on computing square and n^{th} roots over finite fields without requiring quadratic nonresidues.  UMD Department of Co... +1
- **Primality Proving:** Developed deterministic primality-proving algorithms specifically for Proth numbers.  arXiv

Distributed Computing and Pi Records

- **Pi Computations:** Leveraged Apache Hadoop infrastructure at Yahoo to compute distant base-16 digits of π , famously identifying that the two quadrillionth bit of π is 0.
- **Distributed Algorithms:** Researched large-scale integer multiplication via ³ MapReduce variants of the Schönhage–Strassen algorithm.  ResearchGate +4

Contents



1. A Brief Introduction of The **Raft Consensus Algorithm**
2. A Brief Introduction of **Apache Ratis**
3. **The Consistency Model**
 - Algorithms, APIs & Configurations



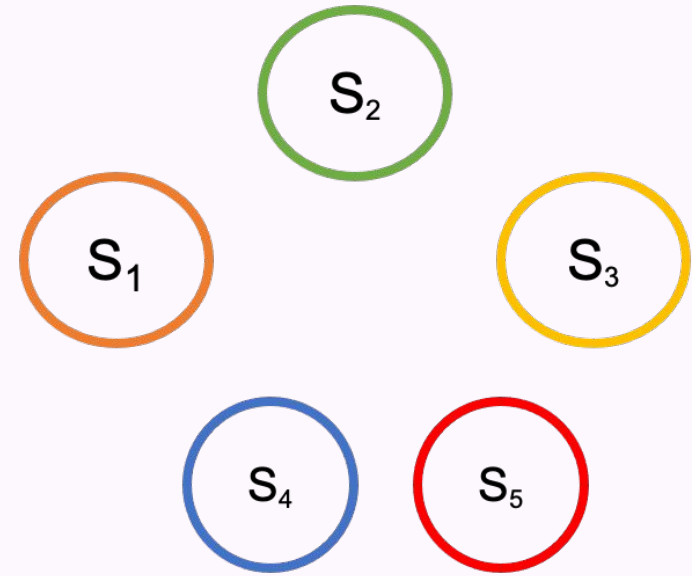
Contents

1. A Brief Introduction of The **Raft Consensus Algorithm**
2. A Brief Introduction of **Apache Ratis**
3. The **Consistency Model**
 - Algorithms, APIs & Configurations



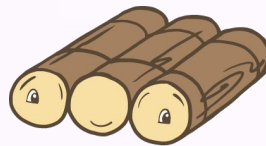
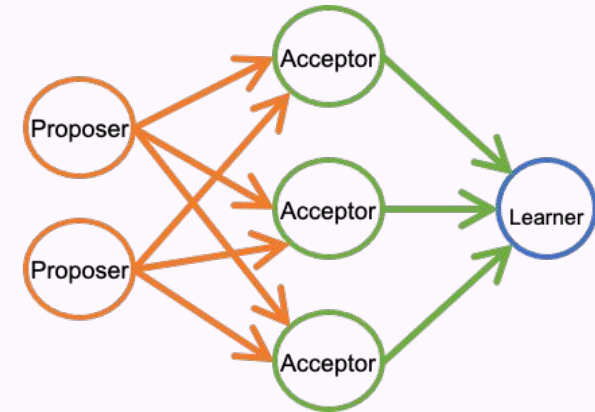
Consensus

- What is *consensus*?
 - Multiple servers to agree a value
- Typical use cases
 - Replicated state machines (high availability)
 - Log replication



Consensus Algorithms

- **Paxos** (1990) – *“The Part-Time Parliament”*
 - Works but hard to understand
 - Hard to implement (correctly)
- **Raft** (2014) – *“In Search of an Understandable Consensus Algorithm”*
 - Easy to understand
 - Easy to prove
 - Easy to implement



Raft Basic

- **Leader Election**
 - RPC: `requestVote`
- **Log Replication**
 - RPC: `appendEntries`

See also <https://raft.github.io/>



Raft Basic – Leader Election

RPC: `requestVote`

- Servers are started as **Followers**
- *Randomized timeout* to become a **Candidate** and start a *leader election*
 - Candidates send `requestVote` to the other servers
 - A server votes for a Candidate only if the candidate has a *up-to-date state*
 - Each server only votes for *at most one* Candidate in each term
- The Candidate becomes the **Leader** once it gets a **majority** of the votes
 - *At most one* Candidate can get a majority
 - Impossible to have two valid Leaders at the same time.

Raft Basic – Log Replication

RPC: `appendEntries`

(1) **Clients** send requests to the **Leader**

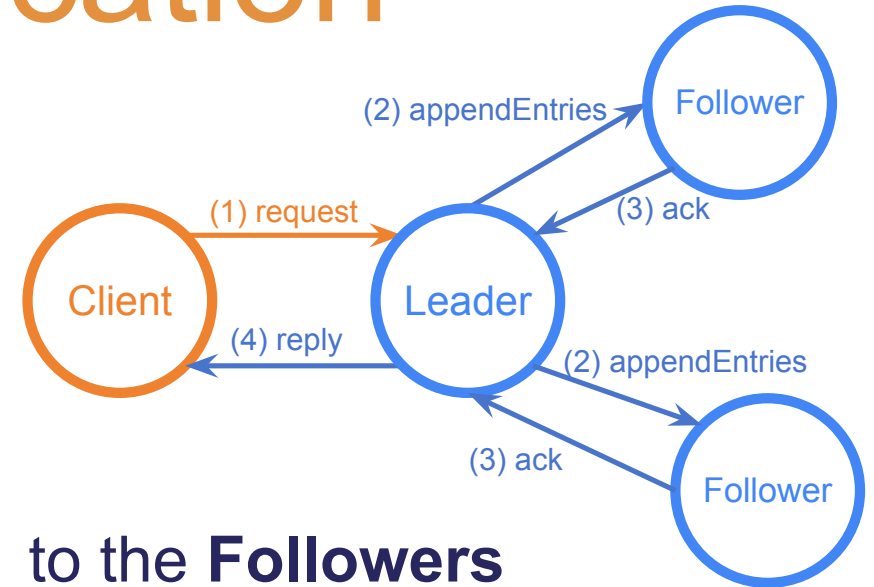
- A non-leader will reject client requests.

(2) Leader forwards client requests as *log entries* to the **Followers** via `appendEntries`.

- Followers *acks* back to the Leader.

(3) Once the Leader has obtained a **majority** of acks, a request is **committed**

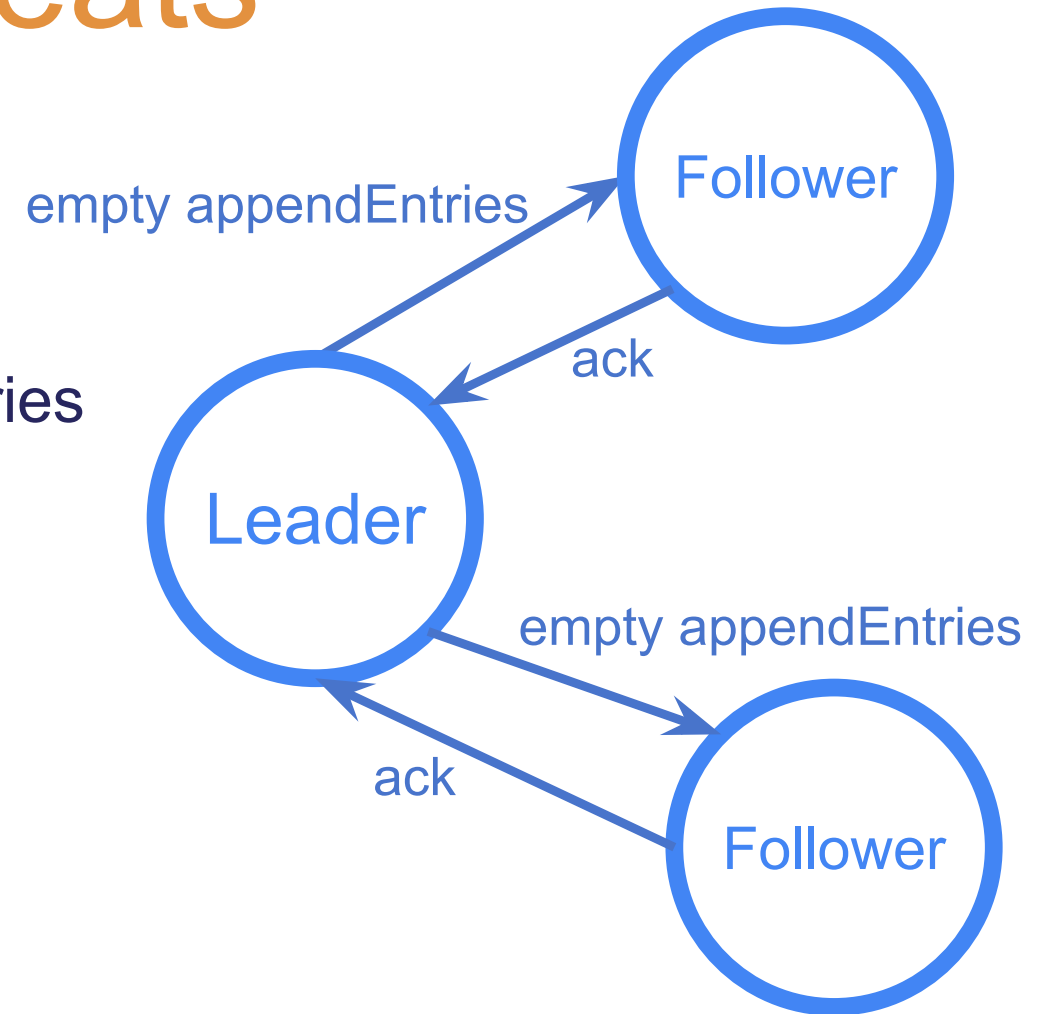
- Committed requests are *safe* regardless of failures or restarts.



Raft Basic – Heartbeats

RPC: `appendEntries`

- **Heartbeat** – empty `appendEntries`
 - i.e. `appendEntries` with zero log entries
- Maintaining leadership
 - Leader must get a **majority** of `appendEntries` acks within *timeout*.
 - Otherwise, the Leader **steps down** to become a follower.



Contents



1. A Brief Introduction of The **Raft Consensus Algorithm**
2. A Brief Introduction of **Apache Ratis**
3. The **Consistency Model**
 - Algorithms, APIs & Configurations



Raft Library

- **Our Motations**
 - To use Raft in *Hadoop Ozone*[†] for HA and Write Pipeline
- **Ratis (2016) – “*In Search of a Usable Raft Library*”** [Raft 2014]
 - A long list of Raft implementations was available but:
 - Most of them were a part of another project.
 - The remaining were just POC or toy implementations.
 - None of them a general library ready to be consumed by other projects
- **We need a Raft library!**

[†] Apache Ozone started as part of Apache Hadoop and later became a standalone project.

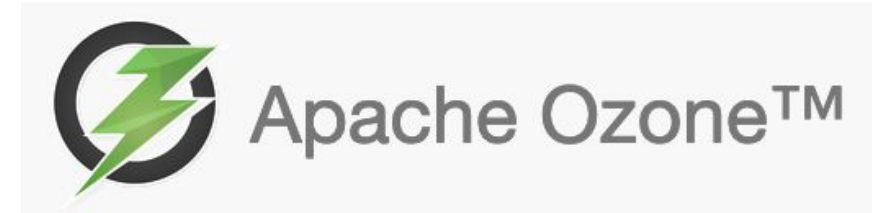
10 Years of Ratis

- 2016-03: Started at *Hortonworks*.
- 2017-01: Entered Apache **Incubation**.
- 2017-04: *Hadoop Ozone* started using Ratis.
- 2017-05: Released first version 0.1.0-alpha.
- 2020-07: Released **first GA** version 1.0.0.
- 2021-02: Graduated as an **ASF Top Level Project**.
- 2021-03: Released the major version 2.0.0.
- 2023-11: Released the major version 3.0.0.
- 2026-04: Released the **latest version 3.2.2**.
- 2026-08: Rolling out 3.3.0 – **Please vote!**



Who uses Apache Ratis?

- **Apache Ozone** (Object Store)
 - OM / SCM high availability
 - Write Pipeline
- **Alluxio** (Data Orchestration for the Cloud)
 - Previously called *Tachyon*
 - Managing all of Alluxio's journaled state
- **Apache IoTDB** (Database for Internet of Things)
 - Replicating application state (HA)
- **Apache Celeborn** (Intermediate Data Service)
 - Replication for high availability



What did Google AI say?

Google
raft java library

✦ AI Overview
+13

Several highly mature, production-grade Java libraries implement the Raft consensus algorithm to build fault-tolerant, strongly consistent distributed systems. Instead of operating as standalone servers like ZooKeeper or Consul, these options function as embeddable libraries directly inside your Java application. Apache Ratis +3

Top Production-Grade Raft Java Libraries

- [Apache Ratis](#): A high-performance, highly customizable Raft library. It separates concerns into pluggable state machines, custom Raft logs, and various RPC implementations like gRPC or Netty. "Ratis aims to make Raft available as a java library that can be used by any system that needs to use a replicated log.", according to [Apache Ratis](#). It scales heavily inside [Apache Ozone](#) and Apache IoTDB. Apache Ratis +4
- [SOFAJRaft](#): An enterprise-grade, full-specification Raft implementation optimized for low latency and extreme throughput. It supports **Multi-Raft-Group** topologies out of the box, making it perfect for heavily loaded distributed key-value databases or partition-based metadata architectures. GitHub

General Use Case of Ratis

- You already have a service running on a **single server**.
- You want to
 - (1) **replicate** the server data to multiple machines
 - The replication number/cluster **membership** can be **changed** in runtime.
 - It can **tolerate server failures**.

or

- (2) have a **HA (highly available)** service
 - When a server fails, another server will **automatically take over**.
 - Clients **automatically failover** to the new server.
- **Apache Ratis is for you!**



Apache Ratis — A Raft Library

- Open Source, Open Development, **Community Driven**
- Apache License 2.0
- Written in Java 8
- Features
 - See “*Apache Ratis - A High Performance Raft Library*” @Community Over Code EU 2024
 - Slides: <https://www.slideshare.net/slideshow/apache-ratis-a-high-performance-raft-library/273692680>
 - Video: <https://www.youtube.com/watch?v=ef9QVG5RSWs>



Asynchronous Event Driven

- Employed:
 - Java **CompletableFuture** & **Executor** APIs
 - **Netty** async event-driven network application framework
 - **gRPC** bi-directional stream API
- **Server-to-server**: asynchronous **appendEntries**
- **Client-to-server**: asynchronous client requests
 - Also support **blocking APIs** in client



Contributions are welcome!

```
static CompletableFuture<?> contributeToApacheRatis(Contributor you) {  
    final CompletableFuture<?> yourFuture =  
        you.submit(yourIdeas -> new PullRequest(you.spendTimeOn(yourIdeas)))  
            .thenAccept(invitation -> new Committer(you, invitation));  
    return yourFuture;  
}
```



Contents

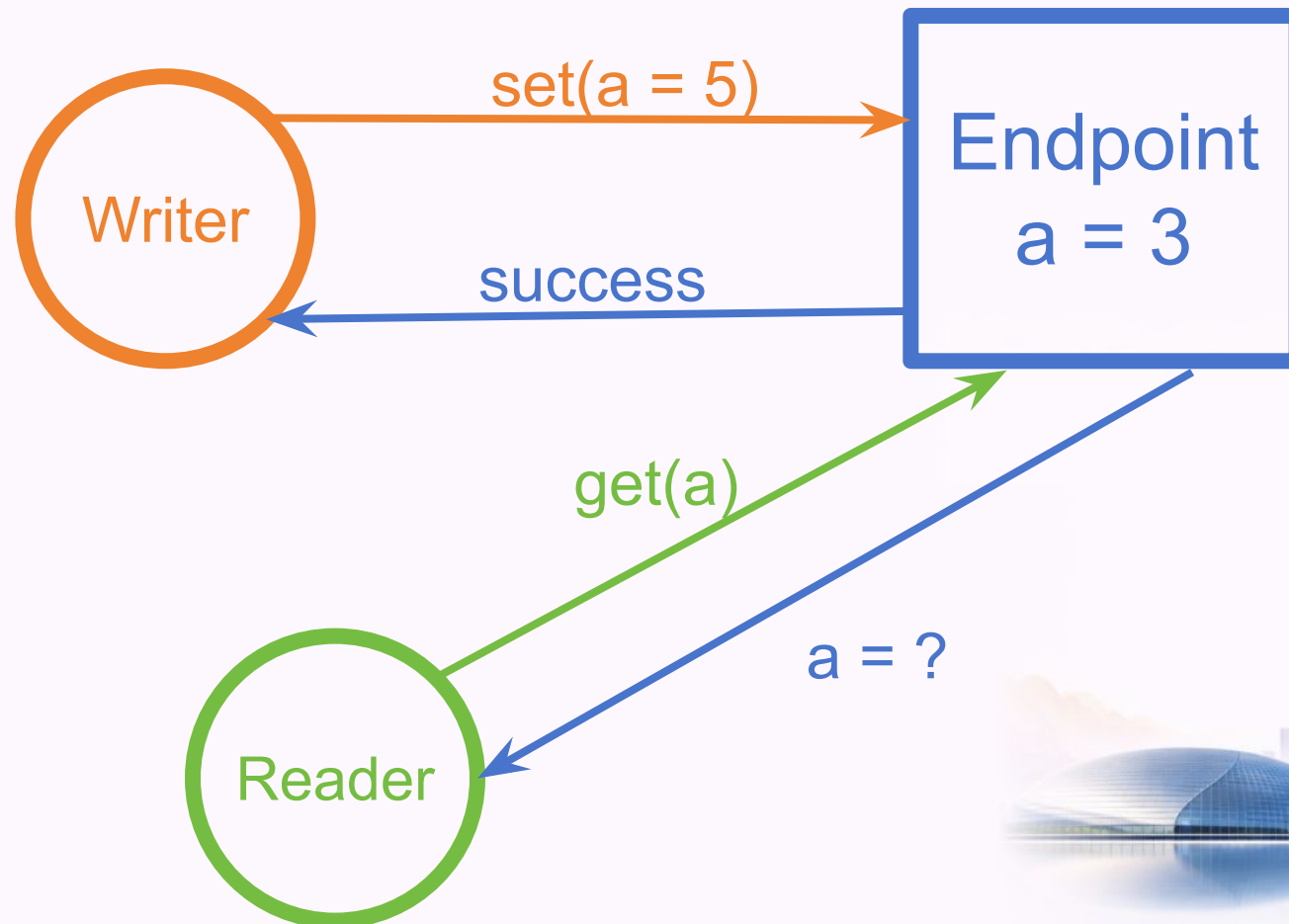


1. A Brief Introduction of The **Raft Consensus Algorithm**
2. A Brief Introduction of **Apache Ratis**
3. **The Consistency Model**
 - Algorithms, APIs & Configurations



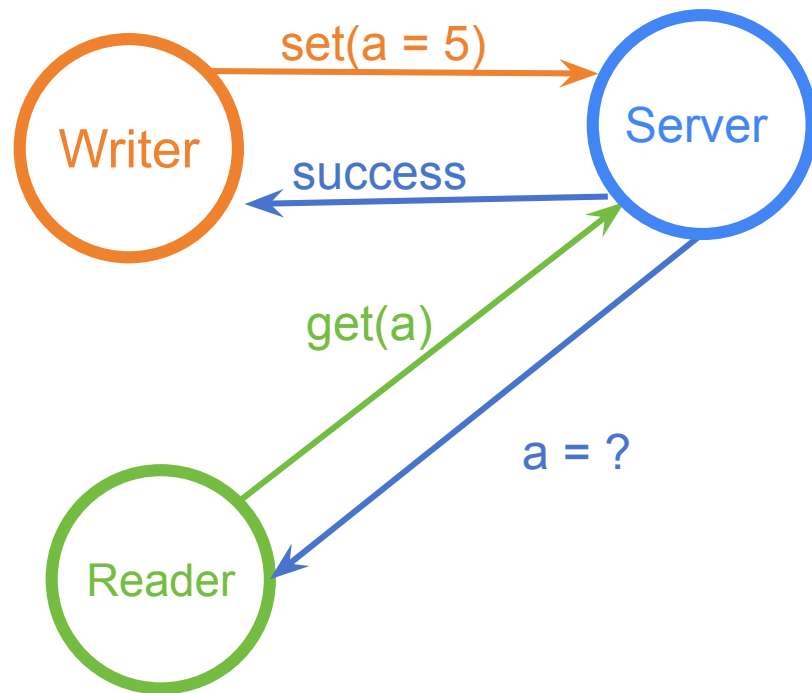
Consistency Model

- Read and write are happening. What to expect?



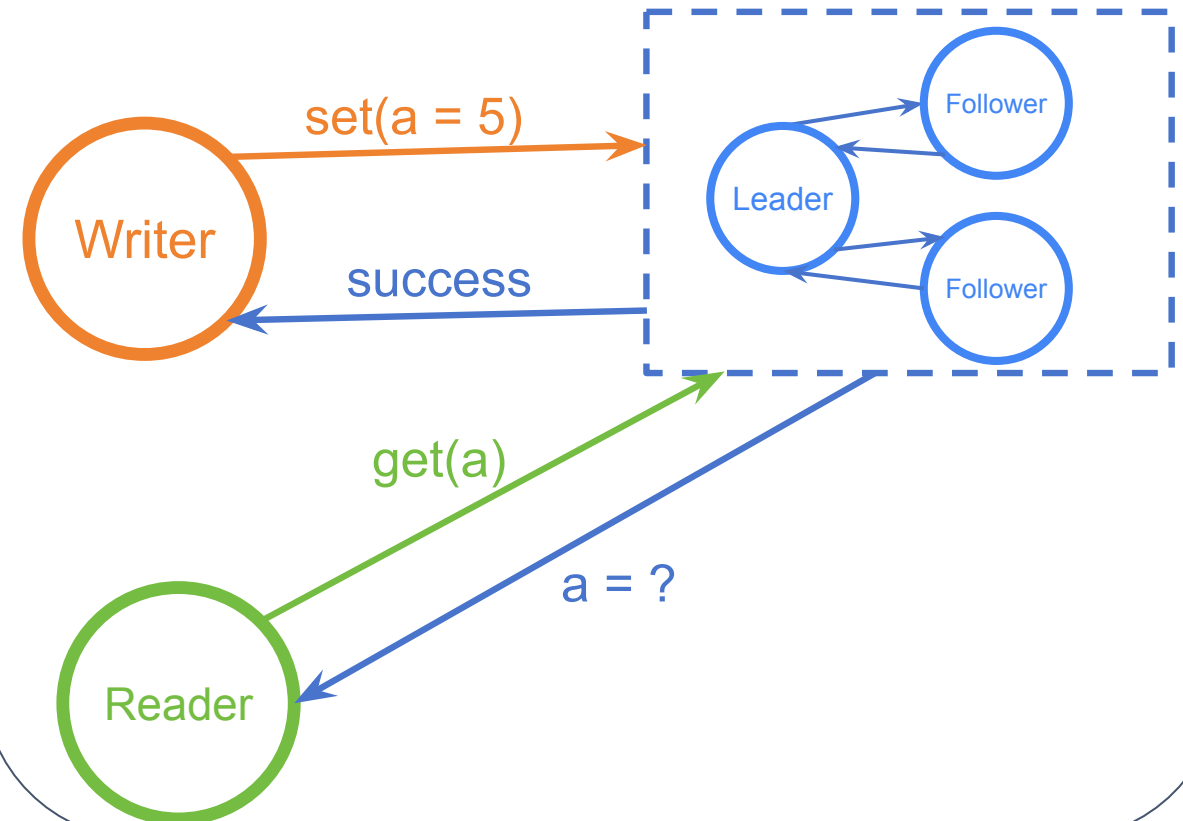
Endpoint ?

A single machine



VS

A group of machines



Consistency Models – Why?

- Why Ratis supports *various consistency models*?

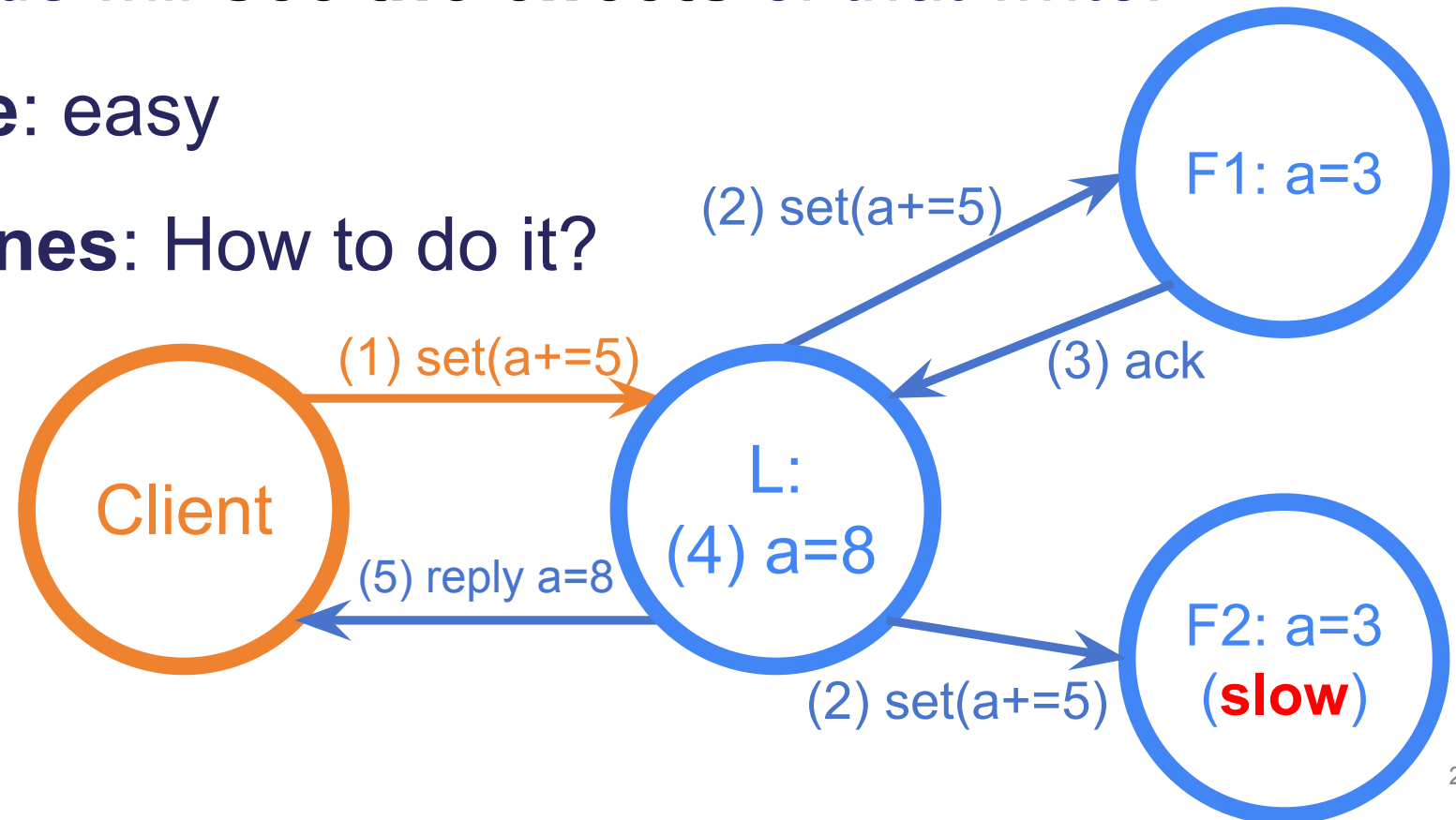
Ratis is a library designed for different use cases:

- **Strictly strong consistency:** Banking systems
- **Mostly strong consistency:** Social networking apps
- **Write demanding:** WAL
- **Read demanding:** Most database apps

Strong Consistency

- Definition: *Once a write operation has been replied as success, all subsequent reads will **see the effects** of that write.*

- **Single machine:** easy
- **Multiple machines:** How to do it?



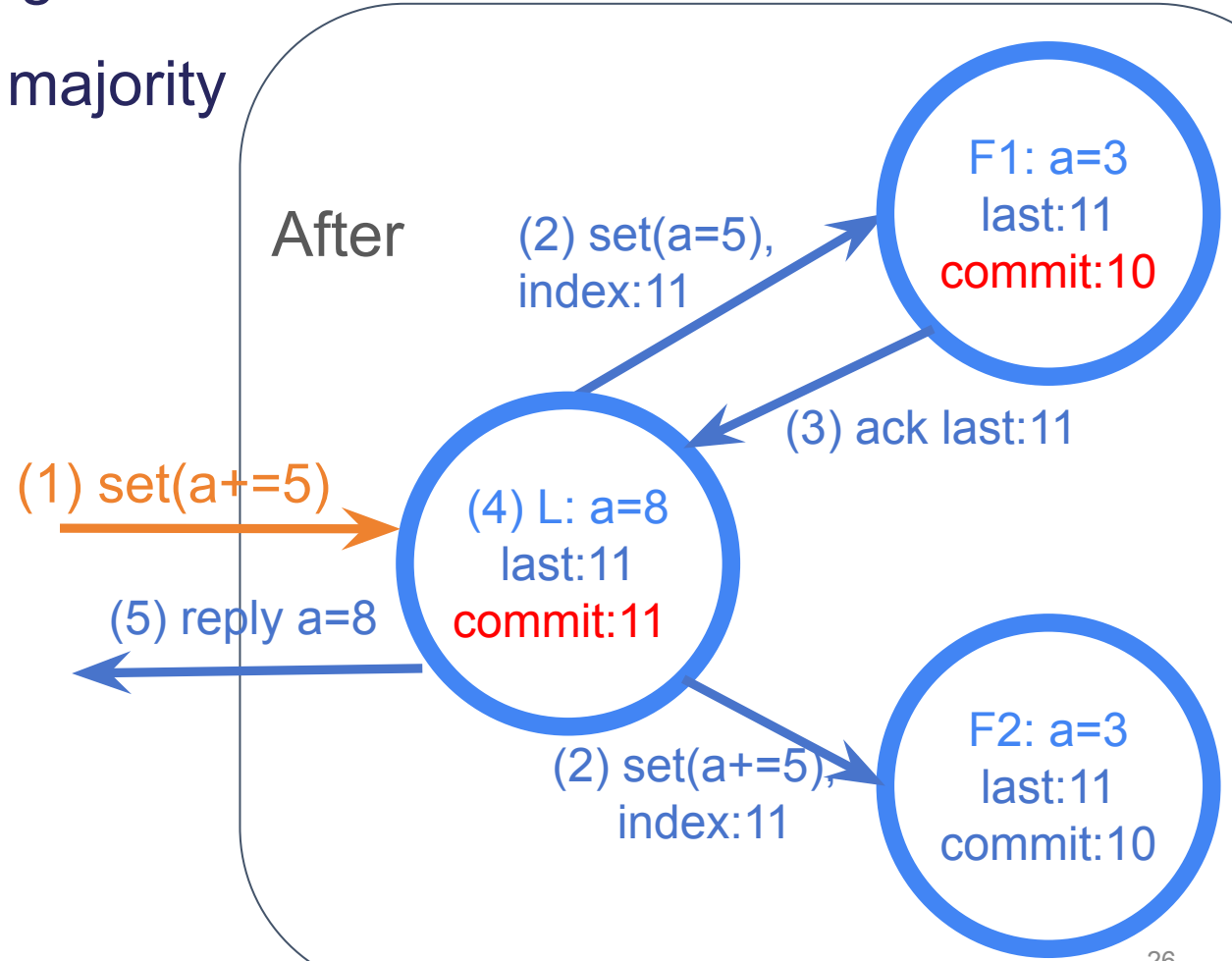
Raft Log Index

- **LastIndex** : the last index of the log
- **CommitIndex**: the index acked by a majority

Before

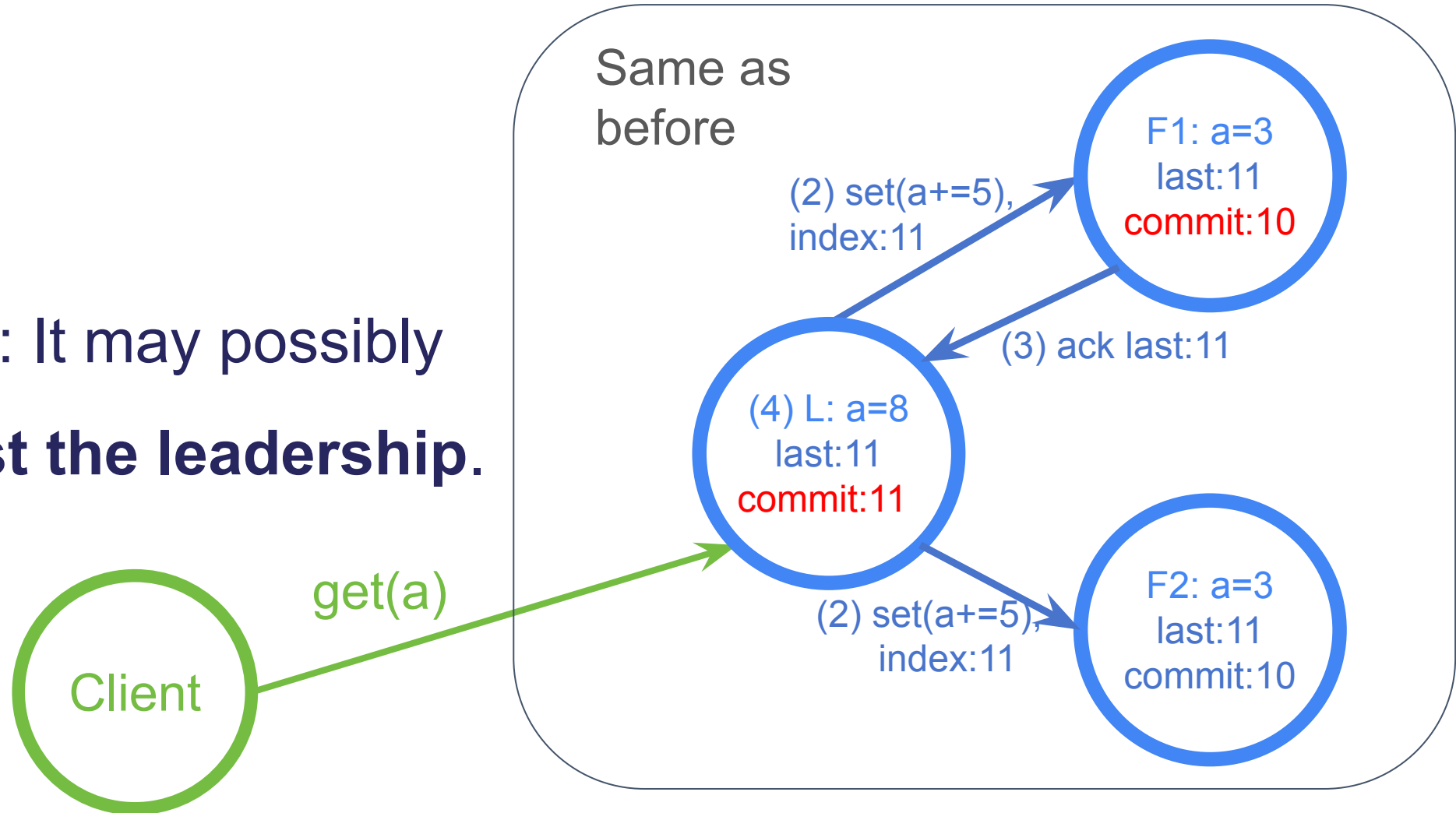


After



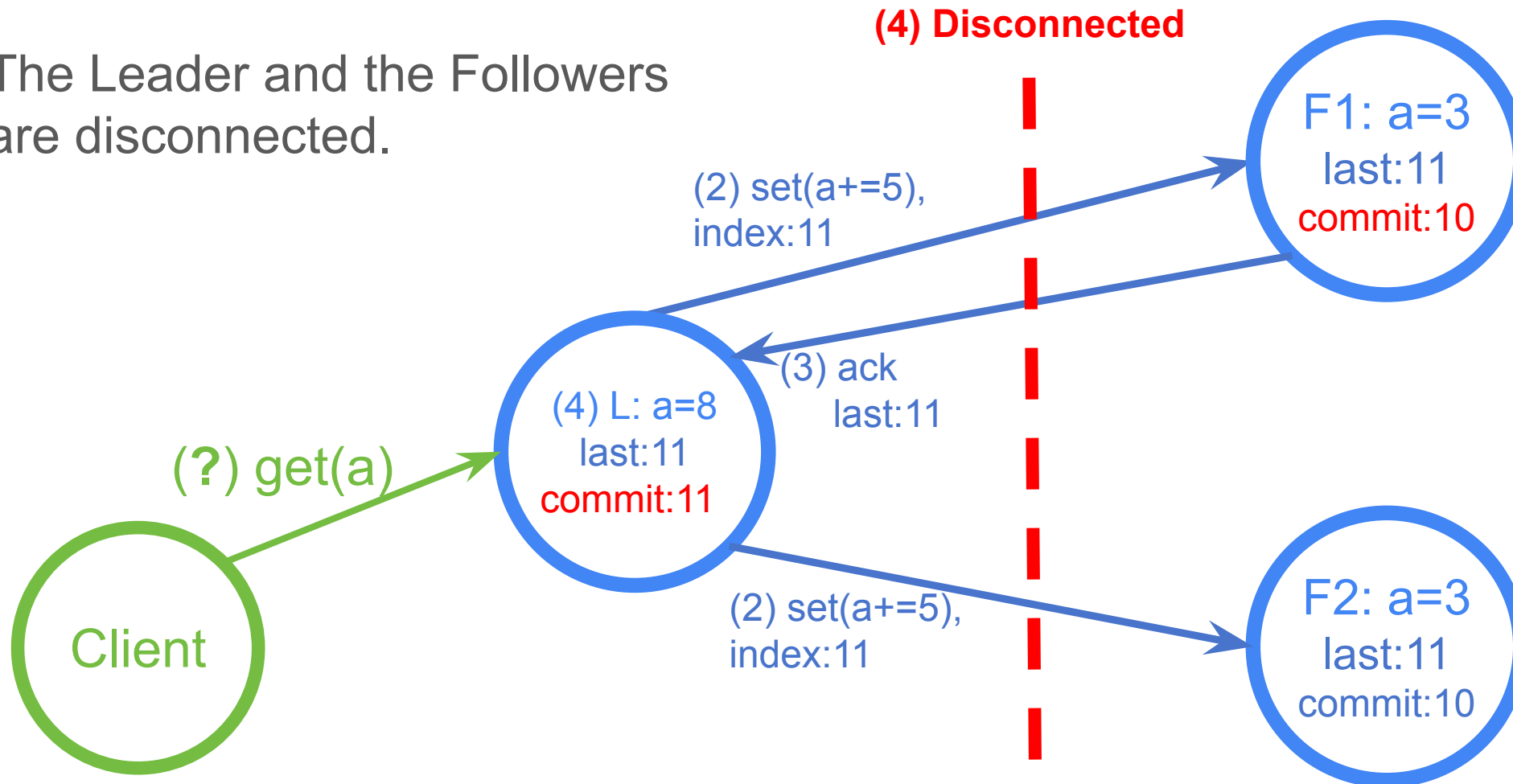
Read from Leader

- It is *safe* ?
 - *Mostly.*
 - Problem: It may possibly have **lost the leadership.**

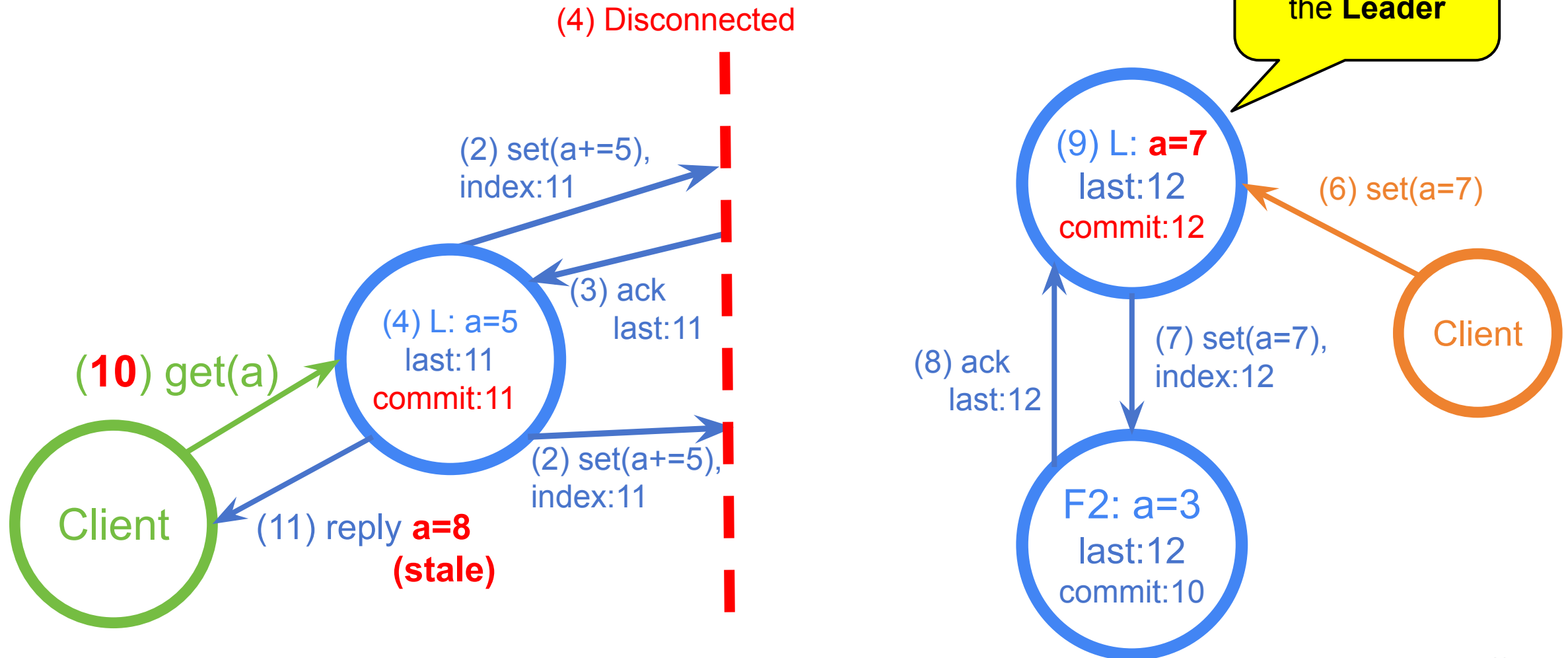


Losing Leadership

The Leader and the Followers are disconnected.

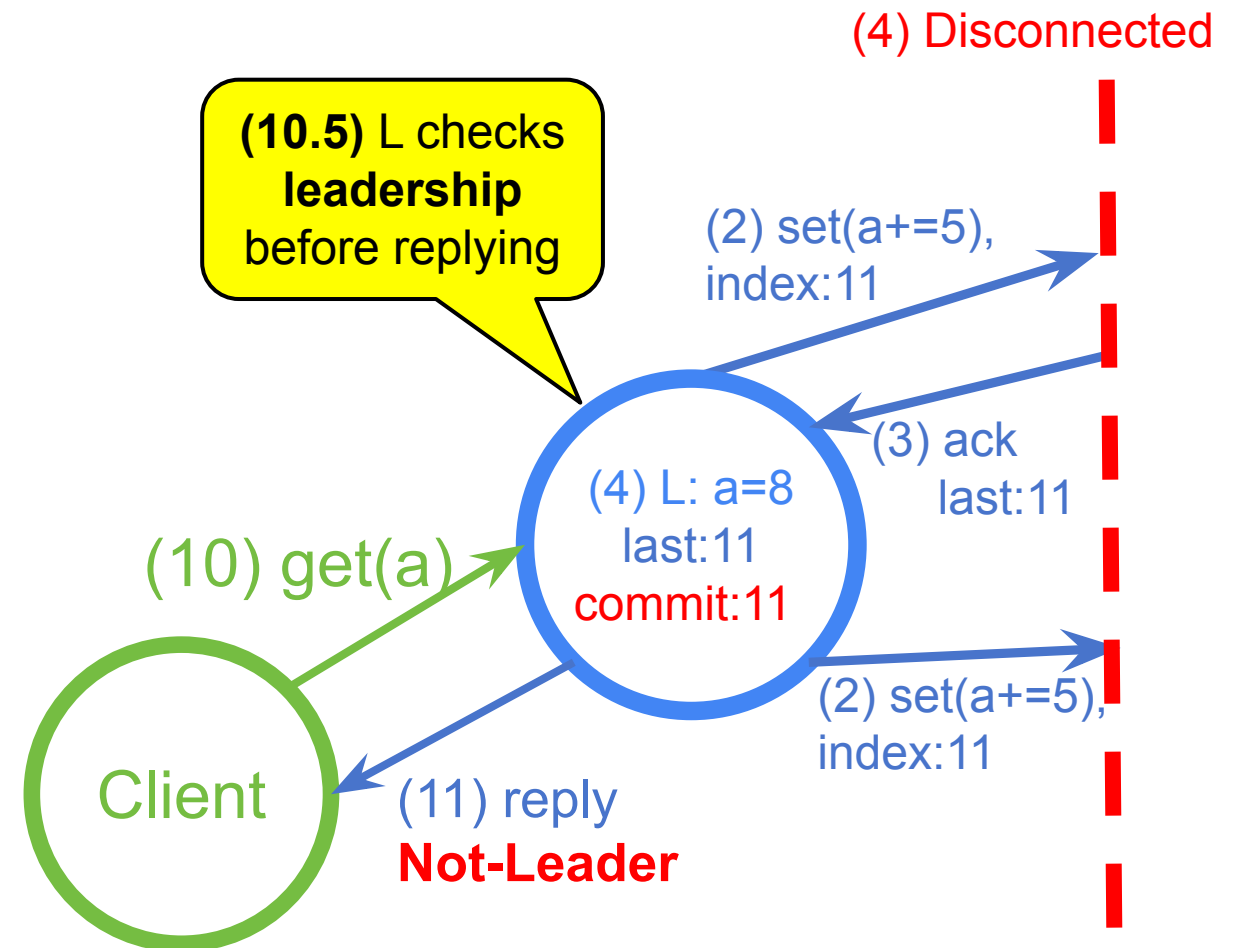


Leader Serving Stale Data



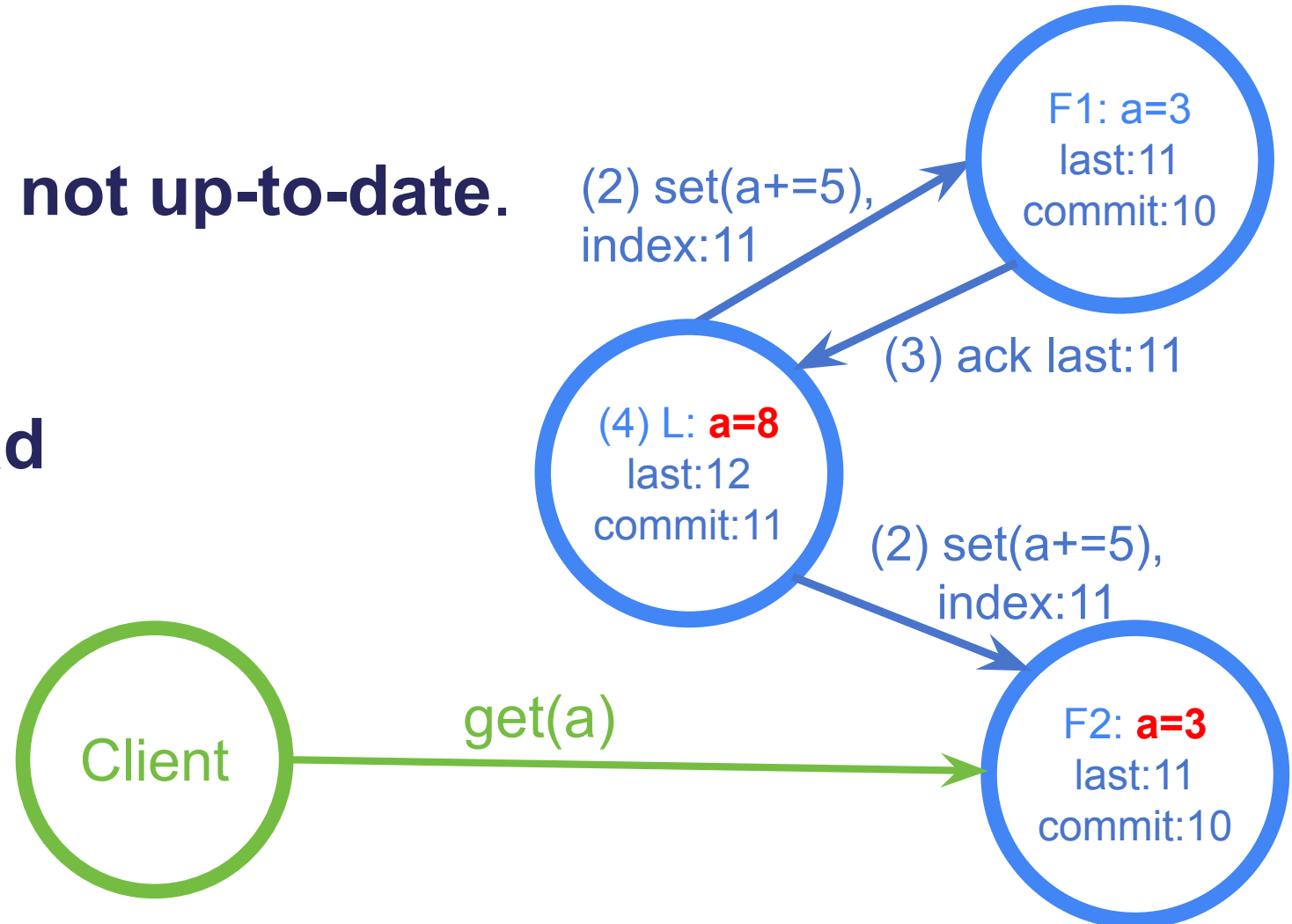
Read from the Leader

- Not **strictly** strong consistency
 - May lose the leadership
 - Before **heartbeat timing out**, it can reply stale data.
- Remedy:
Check leadership before replying
- Trade off:
performance vs *strict* correctness



Read from Followers

- May get stale data
 - Followers may be **not up-to-date**.
- How to avoid it?
 - **Linearizable Read**

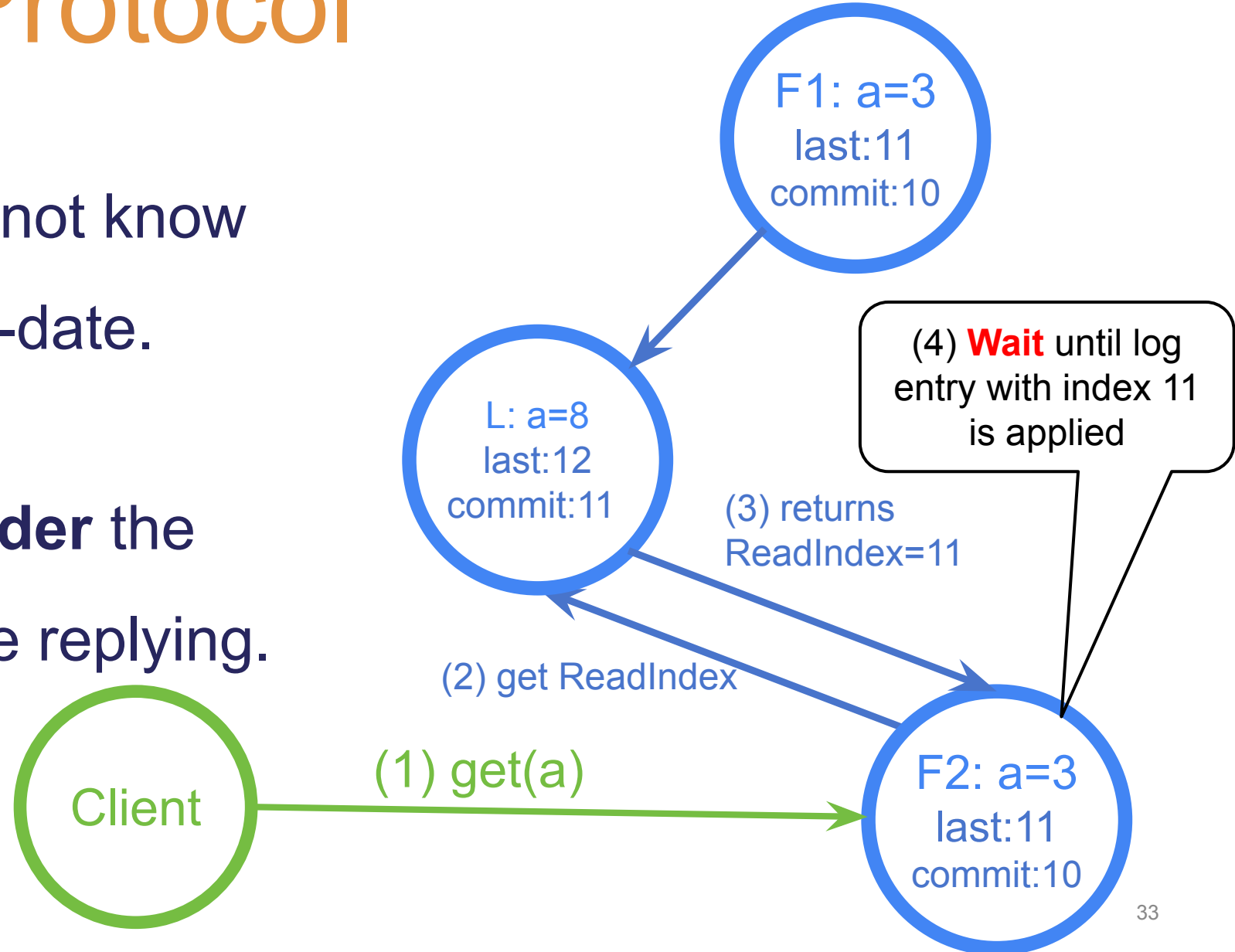


Linearizable Read

- Definition:
 - *Read operations must get the **most up-to-date value** at the time of read.*
- **Leader Linearizable Read:**
 - *Always check leadership before replying (just discussed)*
- **Follower Linearizable Read:**
 - **The ReadIndex Protocol**

ReadIndex Protocol

- Problem:
 - A Follower does not know if its data is up-to-date.
- Solution:
 - Just **ask the Leader** the *ReadIndex* before replying.



ReadIndex

- **LastIndex** : the last index of the **log**
- **CommitIndex** : the index acked by a **majority**
- **AppliedIndex** : the index of the **last applied log entry**
- **RepliedIndex** : the index of the **last replied client request**
- **ReadIndex** : the **CommitIndex** of the **Leader**

Invariant at each server:

LastIndex $\geq$ **CommitIndex** $\geq$ **AppliedIndex** $\geq$ **RepliedIndex**

(write) (commit) (apply) (reply)

One more example

- Suppose
 - **Leader CommitIndex : 20**
 - **Follower AppliedIndex : 15**
- Read from the Follower: The Follower gets ReadIndex=20 and wait for its AppliedIndex. Once it's AppliedIndex $\geq$ 20, it can reply.
- **Quiz:**
 - Is it okay to reply at AppliedIndex $>$ 20, say 25?
 - Certainly – it is the same as the **client request got stuck** in the network.
 - After the Leader replied ReadIndex=20, its CommitIndex advances to 30. Is it still okay for the follower to reply at AppliedIndex=20?
 - Surely – it is the same as the **reply to client got stuck** in the network.

ReadIndex Performance

Problems in the **vanilla** ReadIndex Protocol:

1. Checking **leadership** for every read is expensive!
2. Followers have to **catch up** to the ReadIndex before replying.

Viewing *problems* as **Opportunities**:

1. Avoid checking leadership.
2. Make catching up easier.

Avoid Checking Leadership

Leader Lease [Davis2026]

- Assumption: the network is reasonably fast
- When a follower has ack'ed a heartbeat, it won't timeout for a fraction of the **min timeout** period.
- Leader maintains a *lease*:
 - **Lease** : the max timestamp of a **majority** of servers
 - **Valid** : max timestamp + a fraction of **min timeout**
- When the lease is valid, *assume leadership*.

Make Catching Up Easier (1/3)

- Why using **CommitIndex** as the ReadIndex ?
 - Any indices $>$ CommitIndex are *not committed*
 - Data served after the CommitIndex must not be stale
- The same idea also works for **AppliedIndex** [[RATIS-2379](#)]:
 - Any indices $>$ AppliedIndex are *not applied*
 - Data after the AppliedIndex must not be stale

Idea 1: Since **AppliedIndex** $\leq$ **CommitIndex**, using **AppliedIndex** is easier for the followers to catch up.

Make Catching Up Easier (2/3)

- **RepliedIndex**: the index of the **last replied client request**
 - Any indices $\geq$ **RepliedIndex** are *never visible* to outside.
 - Data after the RepliedIndex can be *considered* as non-stale.
 - [[RATIS-2403](#)]

Idea 2: Since **RepliedIndex** $\leq$ **AppliedIndex**, it is even better to use **RepliedIndex** as the ReadIndex.

Make Catching Up Easier (3/3)

Idea 3: **Batching write replies** (Sacrificing writes for reads)

- *Time-based batching* to send write replies
 - **Coarser** ReadIndex advancement – easier to catch up
- **Write Throughput**: no change
 - No slow down to *applying log entries*.
- **Write Latency**: degraded within a configured time period.
 - The *batching time* is configurable.
- [[RATIS-2403](#)]

Configurations (1/2)

- **Linearizable Read:**

- `raft.server.read.option` (default=DEFAULT)
 - **DEFAULT** : direct read from the Leader
 - **LINEARIZABLE** : linearizable read from Leader or Followers

- **Leader Lease:**

- `raft.server.read.leader.lease.enabled` (default=false)
- `raft.server.read.leader.lease.timeout.ratio` (default=0.9)
 - Lease timeout = (this fraction) * (min timeout)

Configurations (2/2)

- **ReadIndex Choice:**

- `raft.server.read.read-index.type` (default=COMMIT_INDEX)

- COMMIT_INDEX : use commitIndex
- APPLIED_INDEX : use appliedIndex
- REPLIED_INDEX : use repliedIndex

- **Batching Write Replies** (for REPLIED_INDEX only)

- `raft.server.read.read-index.replied-index.batch-interval`

(default=10ms)

- The *time interval* for the time based batching.

Client APIs

- **Async API**

```
// send an async write request
CompletableFuture<RaftClientReply> send(Message message);

// send an async read request
CompletableFuture<RaftClientReply> sendReadOnly(Message message);
```

- **Blocking API** (simply call `CompletableFuture.get()`)

```
// send a blocking write request
RaftClientReply send(Message message) throws IOException;

// send a blocking read request
RaftClientReply sendReadOnly(Message message) throws IOException;
```

Read-after-write Consistency

Consider the following:

1. Initial state: $x = 200$
2. Client sends **async** write: `set(x = 230)`
3. Before (2) completes, it sends **async** read: `get(x)`

The nature of Asynchronous:

- (3) can get *either* $x=200$ or $x=230$.

Read-after-write Consistency [[RATIS-1882](#)]:

- (3) *must* get $x=230$.

Read-after-write API

- **Async only API**

```
// send an async read request with read-after-write consistency
CompletableFuture<RaftClientReply> sendReadAfterWrite(Message message);
```

Note that Blocking API must be read-after-write since the calls are blocking.

- **Configuration: Read-After-Write Cache**

- `raft.server.read.read-after-write-consistent.write-index-cache.expiry-time` (default=60s)
 - Caching the *WriteIndex* for each client at the Leader
 - **WriteIndex**: the index of the **last write request** for a particular client

Stale Read

// async stale read

```
CompletableFuture<RaftClientReply> sendStaleRead(
    Message message, long minIndex, RaftPeerId server);
```

// blocking stale read

```
RaftClientReply sendStaleRead(
    Message message, long minIndex, RaftPeerId server)
    throws IOException;
```

- **minIndex** is the minimum log index already applied in the given server
- The returned data may possibly be outdated (stale).

References

- [Davis2026] A. Jesse Jiryu Davis, Murat Demirbas, and Lingzhi Deng. Leaseguard: Raft leases done right. *Proceedings of the ACM on Management of Data*, 4(1):1–25, April 2026. Available at <https://dl.acm.org/doi/10.1145/3786663>
- [Lamport1998] Leslie Lamport. The part-time parliament. *ACM Trans. Comput. Syst.*, 16(2):133–169, May 1998. Available at <https://dl.acm.org/doi/abs/10.1145/3335772.3335939>
- [Ongaro2014] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319, Philadelphia, PA, June 2014. USENIX Association. Extended version <https://raft.github.io/raft.pdf>
- [Sze2010] Tsz-Wo Sze. The two quadrillionth bit of pi is 0! distributed computation of pi with Apache Hadoop. In *2010 IEEE Second International Conference on Cloud Computing Technology and Science*, pages 727–732, 2010. Available at <https://arxiv.org/abs/1008.3171>
- [Sze2011] Tsz-Wo Sze. Deterministic primality proving on proth numbers, 2011. Available at <https://arxiv.org/abs/2512.15659>





Thanks!

施子 和 Sze, Tsz-Wo (Nicholas)
szetszwo@apache.org

Contributions are welcome!

```
static CompletableFuture<?> contributeToApacheRatis(Contributor you) {  
    final CompletableFuture<?> yourFuture =  
        you.submit(yourIdeas -> new PullRequest(you.spendTimeOn(yourIdeas)))  
            .thenAccept(invitation -> new Committer(you, invitation));  
    return yourFuture;  
}
```